

Spring : Configuration & Auto-Configuration

Maîtriser l'IoC, l'injection de dépendances et les mécanismes d'autoconfiguration de Spring Boot.



Plan de la formation

01

Module 1 – Spring & Spring Boot

Architecture, IoC, autoconfiguration, profils et DevTools.

02

Module 2 – Configuration & Injection

Configuration Java/XML/annotations, cycle de vie des beans, scopes et résolution des dépendances.

03

Mise en pratique

Configurer, injecter, tester et valider des beans Spring.

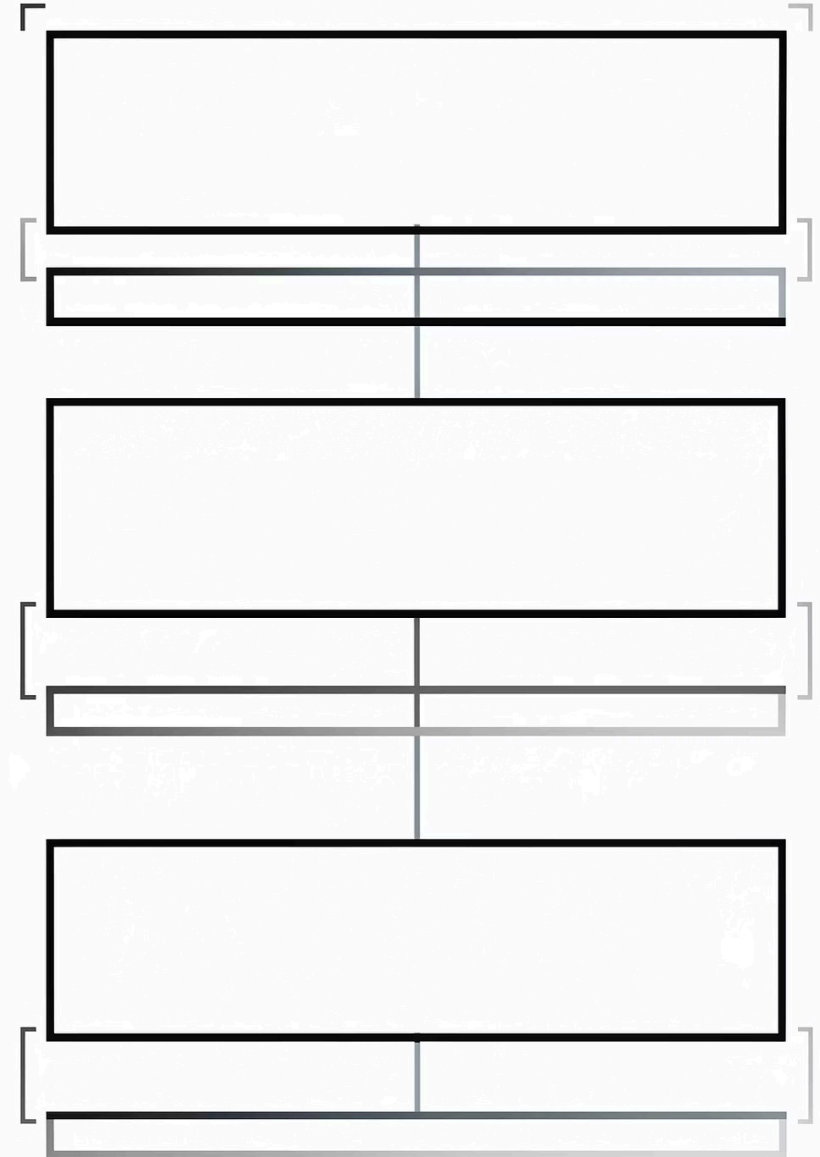
MODULE 1

Introduction à Spring & Spring Boot

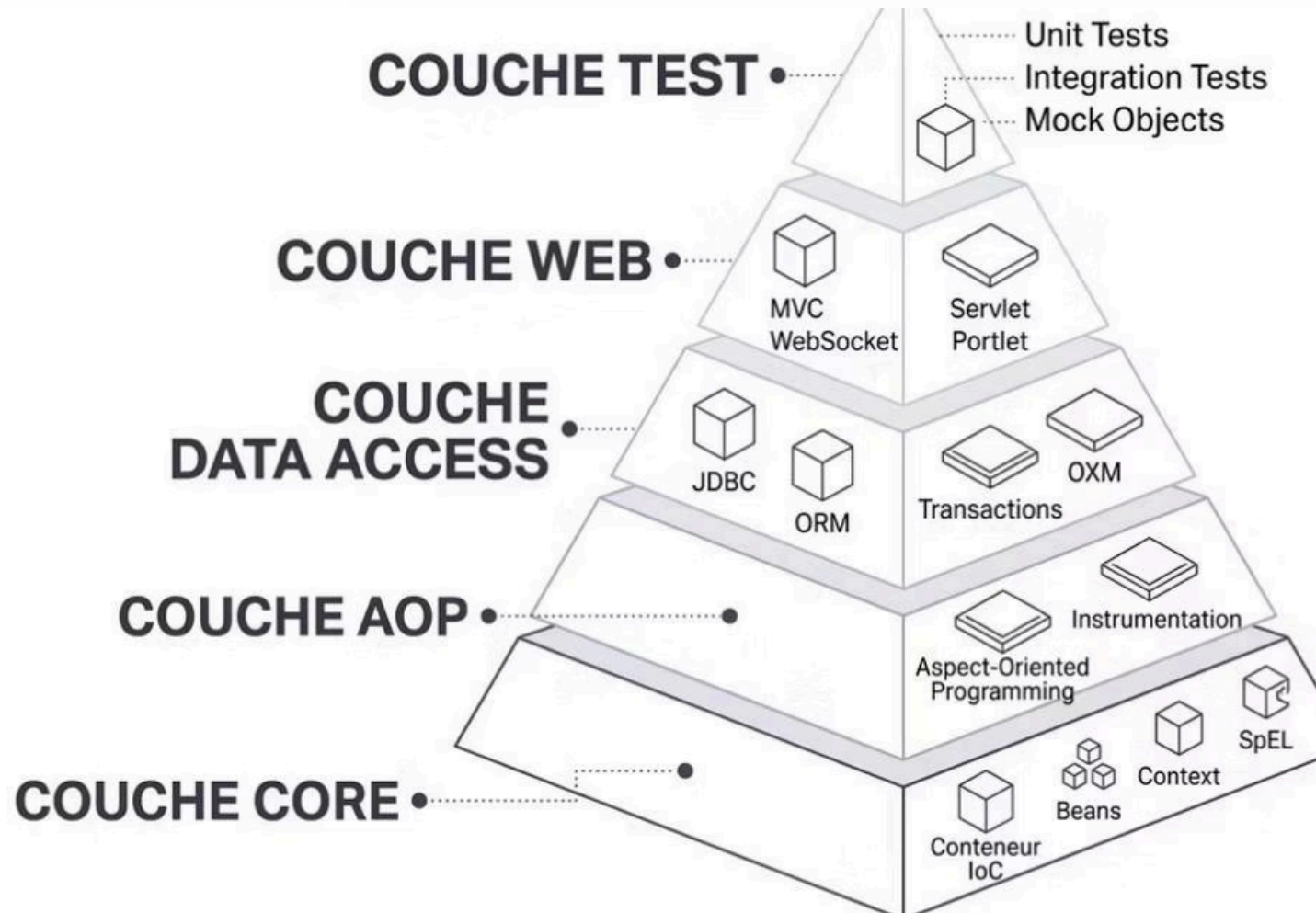
Architecture, IoC, autoconfiguration, profils et bonnes pratiques de développement.

Architecture du Spring Framework

Spring est un écosystème modulaire organisé selon le principe de **séparation des préoccupations**. Au cœur se trouve le **conteneur IoC**, autour duquel gravitent des modules spécialisés : Spring MVC, Spring Data, Spring Security, Spring AOP.



Les couches du Framework



Cinq couches cohérentes : Core (IoC, Beans, SpEL), AOP, Data Access (JDBC, ORM), Web (MVC, WebSocket) et Test. Chaque couche est indépendante — adoptez uniquement les modules dont vous avez besoin.

Les composants principaux



Spring Core

Conteneur IoC, gestion des beans et contexte d'application.
Socle de tout le framework.



Spring Data

Accès aux bases relationnelles et NoSQL via des abstractions puissantes (repositories JPA).



Spring MVC

Framework web MVC pour construire des applications web et des API REST maintenables.



Spring Security

Authentification et autorisation complètes : OAuth2, JWT et bien d'autres protocoles.



Inversion de Contrôle (IoC)

Dans une application Java classique, le code crée et gère ses propres objets. Avec l'IoC, ce contrôle est **inversé** : le conteneur Spring prend en charge la création, la configuration et le câblage des objets.

- i Les classes deviennent plus simples, plus testables — on substitue facilement des implémentations réelles par des mocks.

Sans vs. Avec IoC

✗ Sans injection

Couplage fort, tests difficiles, violation du principe de responsabilité unique.

```
public class OrderService {  
    private final PaymentService p  
        = new PaymentService(); // Fort couplage  
}
```

✓ Avec injection

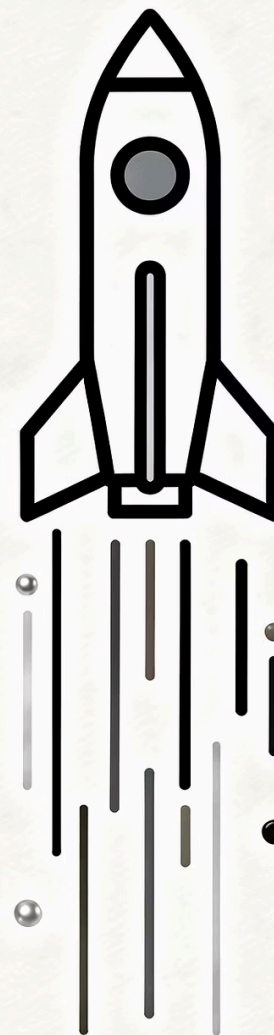
Découplé, testable, maintenable. Le conteneur fournit la dépendance.

```
@Service  
public class OrderService {  
    private final PaymentService p;  
    public OrderService(PaymentService p) {  
        this.payment = p; // Injection constructeur  
    }  
}
```

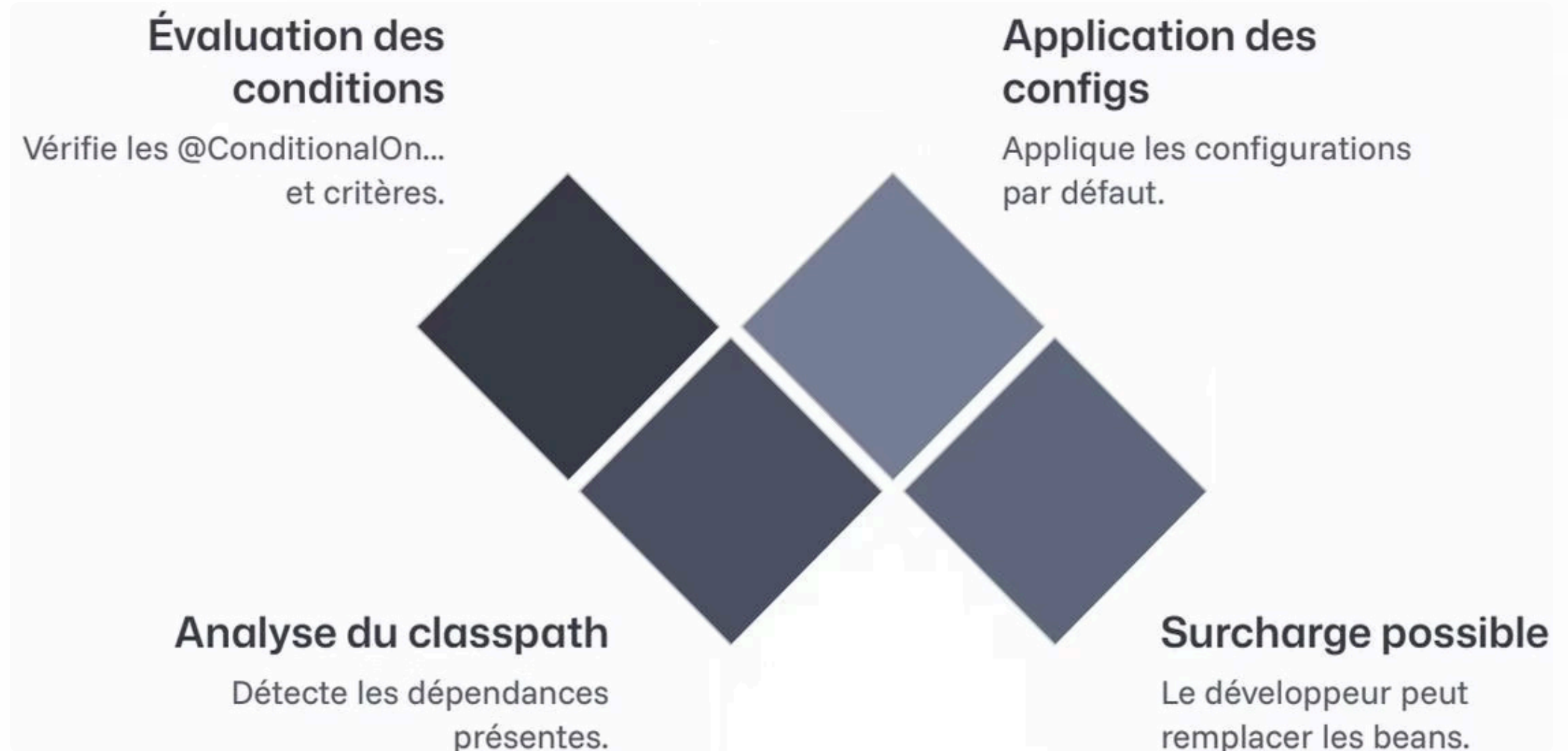

Spring Boot : Convention over Configuration

Lancé en 2014, Spring Boot élimine la friction liée à la configuration. Sa philosophie : **convention over configuration**. Des valeurs par défaut sensées sont appliquées automatiquement — vous ne surchargez que ce qui est nécessaire.

Une application complète avec serveur Tomcat embarqué peut être créée et démarrée en quelques minutes, sans XML ni configuration manuelle.



L'autoconfiguration Spring Boot



Spring Boot analyse le classpath, les beans existants et les propriétés, puis configure automatiquement les composants nécessaires. Si H2 et Spring Data JPA sont détectés, une source de données en mémoire est configurée sans intervention.

Comment fonctionne l'autoconfiguration



Mécanisme clé

Activé par `@EnableAutoConfiguration` (inclus dans `@SpringBootApplication`), les configurations sont déclarées dans :

- `META-INF/spring.factories`
- `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`

Chaque configuration est **conditionnelle** — elle ne s'applique que dans le contexte approprié.

@SpringBootApplication : annotation composite

@Configuration

La classe contient des définitions de beans via des méthodes @Bean.

@EnableAutoConfiguration

Active l'analyse du classpath et la configuration automatique des composants.

@ComponentScan

Détecte automatiquement les composants Spring dans le package courant et ses sous-packages.

Profils et propriétés d'application

Hiérarchie des propriétés

01

Arguments de ligne de commande

02

Variables d'environnement système

03

application-{profile}.properties

04

application.properties **embarqué**

Activer un profil

Via les propriétés :

```
spring.profiles.active=prod
```

Via la JVM :

```
-Dspring.profiles.active=prod
```

Via variable d'environnement :

```
SPRING_PROFILES_ACTIVE=prod
```

Profils Spring : dev vs prod

```
@Configuration
@Profile("dev")
public class DevDataSourceConfig {
    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2).build();
    }
}
```

```
@Configuration
@Profile("prod")
public class ProdDataSourceConfig {
    @Bean
    public DataSource dataSource() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(env.getProperty("db.url"));
        return new HikariDataSource(config);
    }
}
```

 Ne jamais coder en dur des URL, identifiants ou mots de passe. Utilisez toujours des propriétés externalisées ou un gestionnaire de secrets (Vault, AWS Secrets Manager).

Spring Boot DevTools



Redémarrage automatique

Redémarre l'application dès qu'un fichier de classes est modifié, via deux classloaders distincts.



LiveReload

Rafraîchit automatiquement le navigateur lorsque les ressources statiques changent.



Config développement

Caches de templates désactivés, logs plus verbeux — optimisé pour le développement.



Console H2

Active automatiquement la console web H2 lorsqu'une base embarquée est détectée.

Spring Boot Actuator

`/actuator/health`

État de santé de l'application et de ses dépendances (BDD, broker, cache).

`/actuator/metrics`

Mémoire JVM, CPU, requêtes HTTP, connexions BDD et métriques personnalisées.

`/actuator/env`

Toutes les propriétés actives, leur source et valeur (données sensibles masquées).

`/actuator/beans`

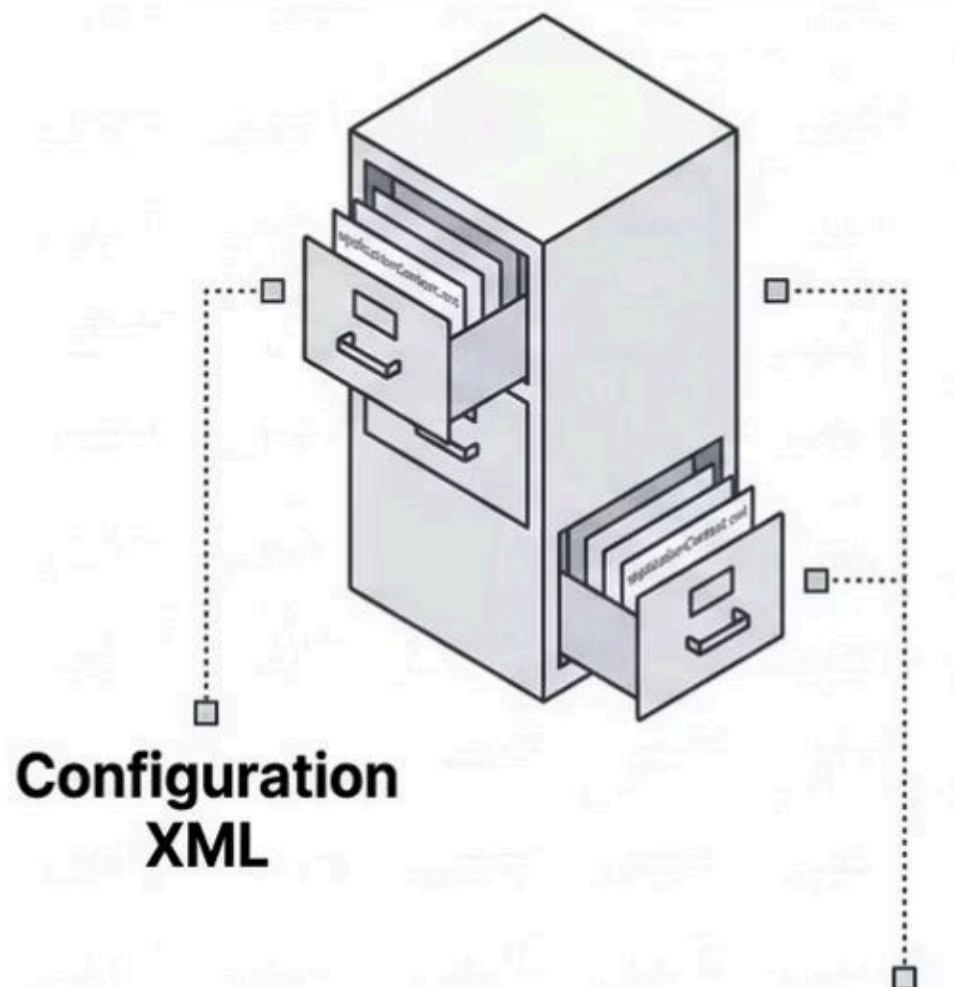
Liste tous les beans du contexte, leurs dépendances et leur scope.

 En production, exposez uniquement les endpoints nécessaires et sécurisez-les avec Spring Security.

Configuration & Injection de dépendances

Trois approches de configuration, cycle de vie des beans, scopes et résolution des dépendances.

Trois approches de configuration



XML – Approche historique

Fichiers `applicationContext.xml`, verbeux, refactoring difficile. Compatible Spring 1.x. Encore présent dans les projets legacy.

Java @Configuration – Approche moderne

Type-safe, refactoring facile, IDE-friendly. Recommandé pour les configurations complexes ou les beans de bibliothèques tierces. Spring 3+.

Annotations – Approche courante

Component scanning automatique. La plus concise et populaire dans les projets Spring Boot modernes. Spring 2.5+.

Configuration XML : l'approche historique

```
<beans xmlns="http://www.springframework.org/schema/beans">
```

```
<!-- Définition d'un bean simple -->
```

```
<bean id="paymentService"  
  class="com.example.PaymentServiceImpl"  
  scope="singleton">  
  <property name="taxRate" value="0.20"/>  
</bean>
```

```
<!-- Injection par constructeur -->
```

```
<bean id="orderService"  
  class="com.example.OrderServiceImpl">  
  <constructor-arg ref="paymentService"/>  
</bean>
```

```
</beans>
```

 La configuration XML manque de type-safety : les erreurs (noms de classes incorrects, propriétés inexistantes) ne sont détectées qu'au démarrage, pas à la compilation.

Configuration Java avec @Configuration

```
@Configuration
public class ApplicationConfig {

    @Bean
    @Scope("singleton") // optionnel, singleton par défaut
    public PaymentService paymentService() {
        PaymentServiceImpl service = new PaymentServiceImpl();
        service.setTaxRate(0.20);
        return service;
    }

    @Bean
    public OrderService orderService() {
        // Spring gère le scope via proxy CGLIB
        return new OrderServiceImpl(paymentService());
    }

    @Bean
    @Profile("prod")
    public AuditService auditService(DataSource dataSource) {
        return new AuditServiceImpl(dataSource);
    }
}
```

- ❗ Quand une méthode `@Bean` en appelle une autre dans la même classe `@Configuration`, Spring intercepte l'appel via un proxy CGLIB et retourne toujours le même bean singleton.

Configuration par annotations

@Component

Annotation générique. Classe de base dont dérivent @Service, @Repository et @Controller.

@Service

Spécialisation pour la couche service (logique métier).
Facilite la lisibilité et la détection ciblée.

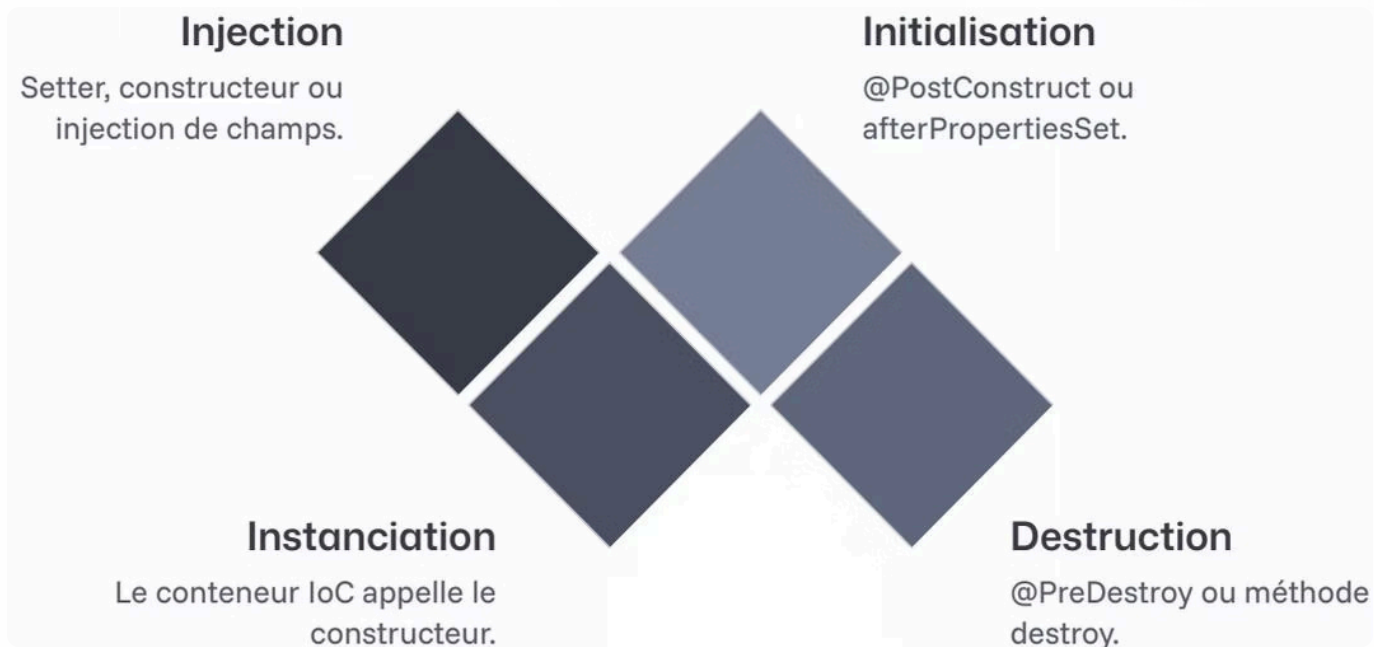
@Repository

Pour les classes d'accès aux données. Spring traduit automatiquement les exceptions de persistance en `DataAccessException`.

@RestController

Combine @Controller et @ResponseBody. Spécialisé pour les API REST.

Le cycle de vie d'un bean Spring



Les quatre étapes

01

Instanciation

Le conteneur IoC appelle le constructeur.

02

Injection

Setter, constructeur ou injection de champs.

03

Initialisation

`@PostConstruct` ou `afterPropertiesSet()`.

04

Destruction

`@PreDestroy` ou méthode `destroy`.

Callbacks @PostConstruct et @PreDestroy

@Service

```
public class DatabaseConnectionPool {
```

```
    private Connection[] pool;
```

@PostConstruct

```
    public void initialize() {
```

```
        // Appelé après injection de toutes les dépendances
```

```
        this.pool = createConnections(10);
```

```
    }
```

@PreDestroy

```
    public void cleanup() {
```

```
        // Appelé avant la destruction du bean
```

```
        for (Connection conn : pool) {
```

```
            closeQuietly(conn);
```

```
        }
```

```
    }
```

```
}
```

✅ Recommandation officielle : utiliser les annotations JSR-250 (@PostConstruct, @PreDestroy) — elles ne créent pas de couplage vers Spring. Toujours libérer les ressources dans @PreDestroy.

Les trois types d'injection

✓ Injection par constructeur – Recommandée

Dépendances obligatoires déclarées dans le constructeur. Permet l'immuabilité (final), facilite les tests, rend les dépendances explicites.

Détecte les dépendances circulaires à la compilation.

⚠ Injection par setter – Optionnelle

Pour les dépendances optionnelles. Permet de reconfigurer après construction, mais au prix de la mutabilité du bean.

✗ Injection par champ – Déconseillée

@Autowired directement sur le champ. Pratique à écrire mais difficile à tester, viole l'encapsulation et masque les dépendances réelles.

Injection par constructeur : la meilleure pratique

@Service

```
public class OrderService {  
  
    private final PaymentService paymentService;  
    private final InventoryService inventoryService;  
    private final NotificationService notificationService;  
  
    // @Autowired optionnel depuis Spring 4.3 (constructeur unique)  
    public OrderService(  
        PaymentService paymentService,  
        InventoryService inventoryService,  
        NotificationService notificationService) {  
        this.paymentService = paymentService;  
        this.inventoryService = inventoryService;  
        this.notificationService = notificationService;  
    }  
}
```

- ❗ Le mot-clé `final` sur les champs garantit l'immuabilité du bean après construction et permet au compilateur de détecter toute omission d'initialisation.

Injection par setter : les cas d'usage

```
@Service
public class ReportService {

    private final DataService dataService; // Obligatoire → constructeur

    private EmailService emailService; // Optionnelle → setter

    public ReportService(DataService dataService) {
        this.dataService = dataService;
    }

    @Autowired(required = false)
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }

    public Report generateReport(String id) {
        Report report = dataService.fetchData(id);
        if (emailService != null) {
            emailService.sendReport(report);
        }
        return report;
    }
}
```

Résolution des beans avec @Autowired

Résolution par type

Si un seul bean du type requis existe, Spring l'injecte directement.

```
@Autowired
private PaymentService paymentService;
// Fonctionne si un seul bean
// PaymentService existe
```

Résolution par nom avec @Qualifier

Si plusieurs beans du même type existent, @Qualifier désambiguïse.

```
@Autowired
@Qualifier("stripePayment")
private PaymentService paymentService;
// Injecte le bean nommé
// "stripePayment"
```

 Vous pouvez aussi utiliser @Primary pour désigner un bean par défaut parmi plusieurs candidats.

Dépendances circulaires

Le problème

Quand deux beans se dépendent mutuellement, Spring lève une `BeanCurrentlyInCreationException` avec l'injection par constructeur. C'est un signal d'architecture problématique.

Les solutions

→ Restructurer l'architecture

Extraire la responsabilité commune dans un troisième service. Solution recommandée.

→ Utiliser `@Lazy`

Crée un proxy différé. Brise le cycle mais masque le problème sous-jacent.

→ Injection via `ApplicationContext`

Récupérer le bean via `getBean()`. En dernier recours uniquement.

Les scopes des beans Spring



Le scope détermine combien d'instances le conteneur crée et gère. Choisir le bon scope est crucial pour la performance, la cohérence des données et la sécurité des threads.

Scope singleton : thread-safety

✓ Bean singleton thread-safe

```
@Service
public class CalculatorService {
    // Pas d'état mutable → thread-safe
    public BigDecimal calculate(
        BigDecimal a, BigDecimal b) {
        return a.add(b);
    }
}
```

✗ Bean singleton dangereux

```
@Service
public class CounterService {
    // État mutable partagé
    // entre tous les threads !!
    private int count = 0;

    public int increment() {
        return ++count; // Race condition!
    }
}
```

⚠ Utilisez `AtomicInteger` ou `ConcurrentHashMap` pour l'état partagé, ou envisagez le scope `prototype`.

Scope prototype : une instance par demande

```
@Component
@Scope("prototype") // Nouvelle instance à chaque injection
public class ShoppingCart {
    private final List<CartItem> items = new ArrayList<>();

    public void addItem(CartItem item) { items.add(item); }
    public BigDecimal getTotal() {
        return items.stream().map(CartItem::getPrice)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}

// Piège : si CheckoutService est singleton,
// il ne recevra qu'une seule instance de cart !
// Solution : utiliser ObjectFactory

@Service
public class ReportGenerator {
    @Autowired
    private ObjectFactory<ShoppingCart> cartFactory;

    public void process() {
        ShoppingCart cart = cartFactory.getObject(); // Nouvelle instance
    }
}
```

 Spring ne gère pas la destruction des beans prototype. La mémoire est à la charge du garbage collector Java.

Scopes web : request et session

```
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST,
    proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestContext {
    private String requestId = UUID.randomUUID().toString();
    private long startTime = System.currentTimeMillis();
    // ...
}
```

```
@Component
@Scope(value = WebApplicationContext.SCOPE_SESSION,
    proxyMode = ScopedProxyMode.TARGET_CLASS)
public class UserPreferences {
    private String language = "fr";
    private String theme = "dark";
    // ...
}
```

❗ `proxyMode = ScopedProxyMode.TARGET_CLASS` est indispensable lorsqu'un bean web-scoped est injecté dans un singleton : Spring crée un proxy qui délègue vers l'instance du scope approprié.

@Primary et @Qualifier : désambiguïser

```
public interface NotificationService { void send(String message); }
```

```
@Service
```

```
@Primary // Utilisé par défaut
```

```
public class EmailNotificationService implements NotificationService { }
```

```
@Service
```

```
@Qualifier("sms")
```

```
public class SmsNotificationService implements NotificationService { }
```

```
@Service
```

```
public class AlertService {
```

```
    public AlertService(
```

```
        NotificationService defaultNotification, // Email (Primary)
```

```
        @Qualifier("sms") NotificationService smsNotification) {
```

```
        // ...
```

```
    }
```

```
}
```

Conditions d'autowiring avec @ConditionalOn

@ConditionalOnProperty

Bean créé uniquement si une propriété a une valeur spécifique.

```
@ConditionalOnProperty("feature.audit.enabled")
```

@ConditionalOnClass

Bean créé si une classe est présente sur le classpath. Utilisé massivement par l'autoconfiguration Spring Boot.

@ConditionalOnMissingBean

Bean créé uniquement si aucun bean du type spécifié n'existe déjà. Permet aux utilisateurs de surcharger la configuration par défaut.

@ConditionalOnProfile

Équivalent à @Profile. Bean créé uniquement si un profil spécifique est actif.

@ConfigurationProperties : configuration typée

```
@ConfigurationProperties(prefix = "app.mail")
@Validated
public class MailProperties {

    @NotBlank
    private String host;

    @Min(1) @Max(65535)
    private int port = 587;

    @NotBlank
    private String username;

    private boolean ssl = true;
    // Getters et setters
}
```

```
# application.properties
app.mail.host=smtp.example.com
app.mail.port=587
app.mail.username=noreply@example.com
app.mail.password=${MAIL_PASSWORD}
app.mail.ssl=true
```

✔ Utilisez `@ConfigurationProperties` avec des records Java (Java 16+) pour une configuration concise et immutable. Ajoutez `spring-boot-configuration-processor` pour la complétion IDE.

@Value vs @ConfigurationProperties

Critère	@Value	@ConfigurationProperties
Nombre de propriétés	1-2 propriétés isolées	Groupe de propriétés liées
Type-safety	Conversion basique	Typage fort, types complexes
Validation	Manuelle	Bean Validation (@NotNull, etc.)
Complétion IDE	Limitée	Excellente avec processor
Refactoring	Difficile (strings)	Facile (Java types)
Cas d'usage idéal	Propriété unique, simple	Configuration d'un module entier

BeanFactory vs ApplicationContext

BeanFactory

- Chargement paresseux
- Fonctionnalités minimales
- Faible consommation mémoire
- Usage : environnements très contraints

ApplicationContext

- Chargement eager (détection rapide des erreurs)
- Support AOP et transactions
- Internationalisation (i18n)
- Événements applicatifs
- Support Web & Spring Boot

✅ Dans la quasi-totalité des applications modernes, vous utiliserez exclusivement `ApplicationContext`.

Événements applicatifs Spring

```
// Définition d'un événement
public class OrderCreatedEvent extends ApplicationEvent {
    private final Order order;
    public OrderCreatedEvent(Object source, Order order) {
        super(source); this.order = order;
    }
}

// Publication depuis un service
@Service
public class OrderService {
    @Autowired
    private ApplicationEventPublisher publisher;

    public Order createOrder(OrderRequest request) {
        Order order = processOrder(request);
        publisher.publishEvent(new OrderCreatedEvent(this, order));
        return order;
    }
}

// Écoute dans un autre bean
@Service
public class AuditService {
    @EventListener
    public void onOrderCreated(OrderCreatedEvent event) {
        log.info("Commande créée: {}", event.getOrder().getId());
    }
}
```

BeanPostProcessor : personnaliser le cycle de vie

```
@Component
public class PerformanceMonitoringPostProcessor
    implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(
        Object bean, String beanName) throws BeansException {
        System.out.println("Avant init: " + beanName);
        return bean; // Retourner le bean (ou un proxy)
    }

    @Override
    public Object postProcessAfterInitialization(
        Object bean, String beanName) throws BeansException {
        System.out.println("Après init: " + beanName);
        return bean;
    }
}
```

- ❗ Les `BeanPostProcessor` sont instanciés en premier par Spring, avant les autres beans, afin d'être disponibles pour traiter l'ensemble du contexte. C'est la base de l'AOP et de la gestion transactionnelle.

Lazy Loading des beans

```
@Service
@Lazy // Instancié à la première injection
public class HeavyReportService {
    public HeavyReportService() {
        loadReportTemplates();
        initializeChartingEngine();
    }
}

# Lazy loading global (application.properties)
spring.main.lazy-initialization=true
```

✓ Avantages

- Démarrage plus rapide
- Économie de ressources pour les beans rarement utilisés
- Utile en développement et test

⚠ Inconvénients

- Erreurs détectées à la première utilisation, pas au démarrage
- Latence sur le premier appel
- Non recommandé en production pour les services critiques

Injection de collections et de Map

```
public interface PaymentGateway {
    boolean supports(PaymentMethod method);
    PaymentResult process(Payment payment);
}

@Service("stripe")
public class StripeGateway implements PaymentGateway { /* ... */ }

@Service("paypal")
public class PaypalGateway implements PaymentGateway { /* ... */ }

@Service
public class PaymentRouter {
    // Spring injecte TOUTES les implémentations
    // clé = nom du bean, valeur = instance
    private final Map<String, PaymentGateway> gateways;

    public PaymentRouter(Map<String, PaymentGateway> gateways) {
        this.gateways = gateways;
        // {"stripe": StripeGateway, "paypal": PaypalGateway}
    }
}
```

- ❗ Ce pattern permet d'implémenter élégamment Strategy ou Plugin sans modifier le code consommateur lors de l'ajout de nouvelles implémentations.

Ordering des beans avec @Order

@Component

@Order(1) // Priorité la plus haute → exécuté en premier

```
public class StockValidationRule implements ValidationRule {  
    public ValidationResult validate(Order order) { /* ... */ }  
}
```

@Component

@Order(2)

```
public class PaymentValidationRule implements ValidationRule {  
    public ValidationResult validate(Order order) { /* ... */ }  
}
```

@Service

```
public class OrderValidator {  
    private final List<ValidationRule> rules; // Injectées dans l'ordre  
  
    public OrderValidator(List<ValidationRule> rules) {  
        this.rules = rules; // [StockRule(1), PaymentRule(2)]  
    }  
}
```

❗ Les valeurs les plus basses correspondent aux priorités les plus hautes. Essentiel pour les chaînes de filtres, intercepteurs ou validateurs.

Spring Expression Language (SpEL)

@Service

```
public class PricingService {
```

```
// Propriété avec valeur par défaut
```

```
@Value("${pricing.taxRate:0.20}")
```

```
private double taxRate;
```

```
// Accès à un autre bean
```

```
@Value("#{configBean.maxDiscount * 0.5}")
```

```
private double partnerDiscount;
```

```
// SpEL dans @Cacheable
```

```
@Cacheable(value = "prices",
```

```
key = "#product.id + '_' + #currency",
```

```
condition = "#product.price > 100")
```

```
public Price calculatePrice(Product product, String currency) { }
```

```
// SpEL dans Spring Security
```

```
@PreAuthorize("hasRole('ADMIN') or #userId == authentication.name")
```

```
public UserData getUserData(String userId) { }
```

```
}
```

Modulariser avec @Import

```
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }
}

@Configuration
public class CacheConfig {
    @Bean
    public CacheManager cacheManager() {
        return new ConcurrentHashMapCacheManager("users", "products");
    }
}

// Configuration principale qui importe les modules
@Configuration
@Import({SecurityConfig.class, CacheConfig.class})
public class ApplicationConfig {
    @Bean
    public UserService userService(
        PasswordEncoder encoder, // De SecurityConfig
        CacheManager cache) { // De CacheConfig
        return new UserServiceImpl(encoder, cache);
    }
}
```

Erreurs courantes et solutions

NoSuchBeanDefinitionException

Cause : Spring ne trouve pas le bean demandé.

Solution : Vérifier l'annotation (`@Service`, etc.) et que la classe est dans un package scanné par `@ComponentScan`.

NoUniqueBeanDefinitionException

Cause : Plusieurs beans du même type sans désambiguïsation.

Solution : Ajouter `@Primary` sur le bean par défaut ou `@Qualifier` sur le point d'injection.

BeanCurrentlyInCreationException

Cause : Dépendance circulaire avec injection par constructeur.

Solution : Restructurer l'architecture ou utiliser `@Lazy`.

UnsatisfiedDependencyException

Cause : Spring ne peut pas satisfaire une dépendance obligatoire.

Solution : Vérifier les profils actifs, les conditions `@ConditionalOn` et la visibilité des beans.

Checkliste de bonnes pratiques

Configuration

- Utiliser `@ConfigurationProperties` pour les groupes de propriétés
- Externaliser les secrets via variables d'environnement
- Définir au moins 3 profils : dev, test, prod
- Valider les propriétés avec `@Validated`

Injection

- Toujours préférer l'injection par constructeur
- Déclarer les dépendances obligatoires comme `final`
- Utiliser `@Qualifier` pour désambigüiser

Tests

- Inclure un test `contextLoads()` dans chaque module
- Tests unitaires avec Mockito (sans contexte Spring)
- Tests d'intégration avec `@SpringBootTest` et `@ActiveProfiles("test")`

Qualité

- Analyser les dépendances avec `/actuator/beans`
- Vérifier l'absence de dépendances circulaires
- Documenter les scopes non-singleton

Récapitulatif : Module 1

Architecture modulaire

Spring est un écosystème de modules autour d'un conteneur IoC central. Chaque module répond à un besoin précis sans imposer ses dépendances.

IoC & Injection

L'inversion de contrôle délègue la création des objets au conteneur. L'injection par constructeur est l'approche recommandée.

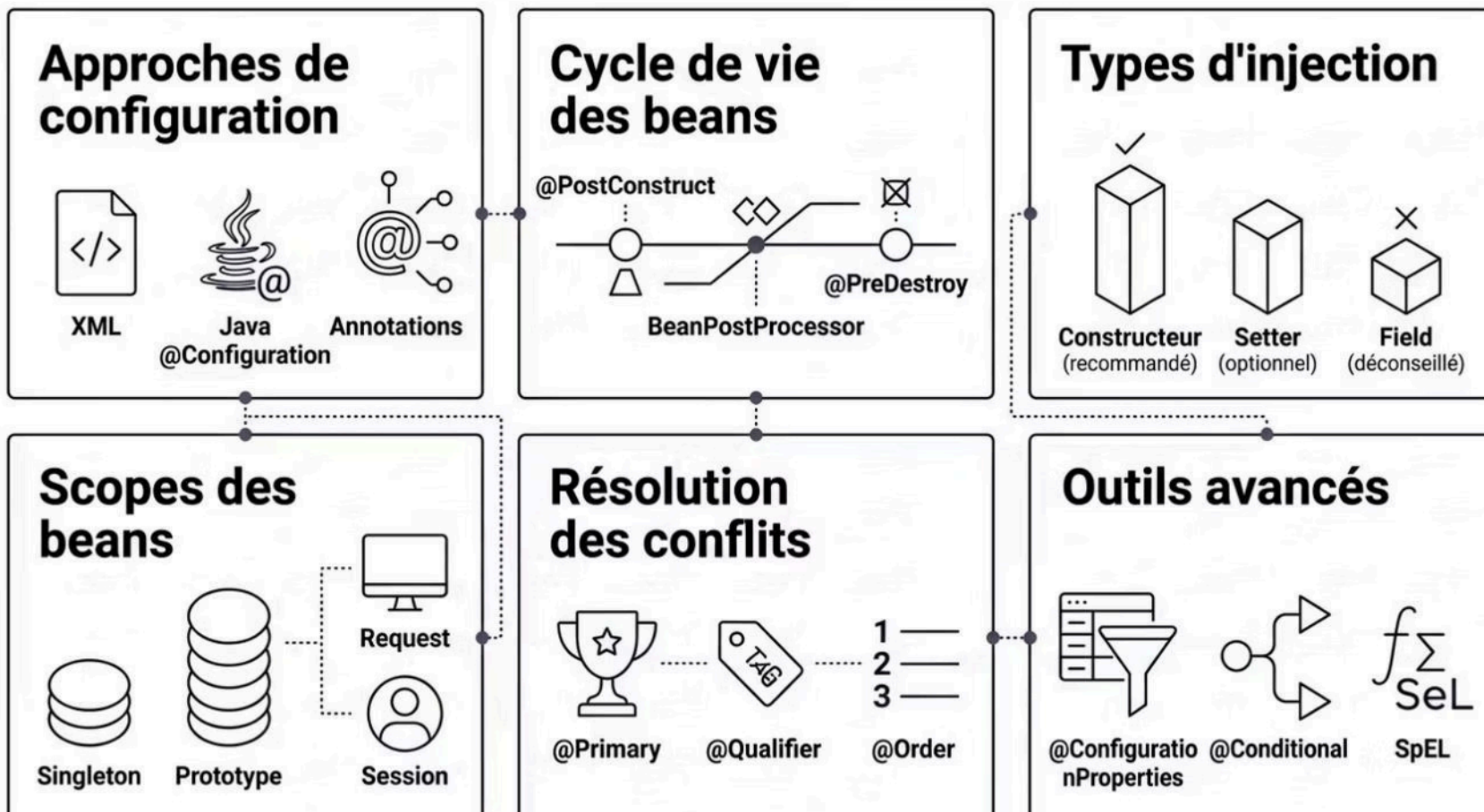
Spring Boot & Autoconfiguration

Élimine la configuration répétitive grâce à l'autoconfiguration conditionnelle. Profils et propriétés externalisées gèrent les environnements.

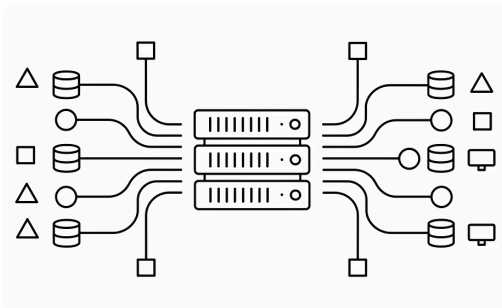
DevTools & Actuators

Hot reload pour le développement. Les Actuators exposent métriques et état de santé essentiels à l'exploitation.

Récapitulatif : Module 2

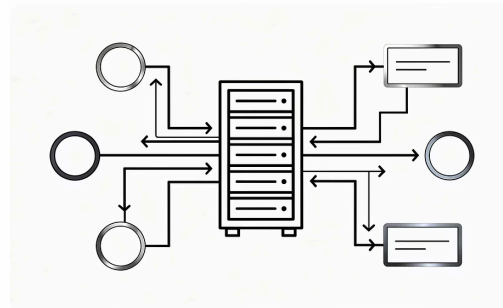


Pour aller plus loin



Spring Data JPA

Repositories, requêtes dérivées, pagination et projections. La couche de persistance simplifiée avec JPA et Hibernate.



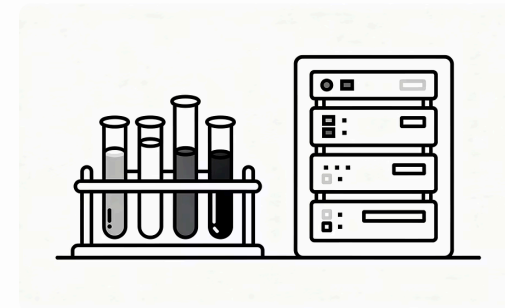
Spring MVC & REST

API RESTful robustes, gestion des erreurs, validation et documentation OpenAPI/Swagger.



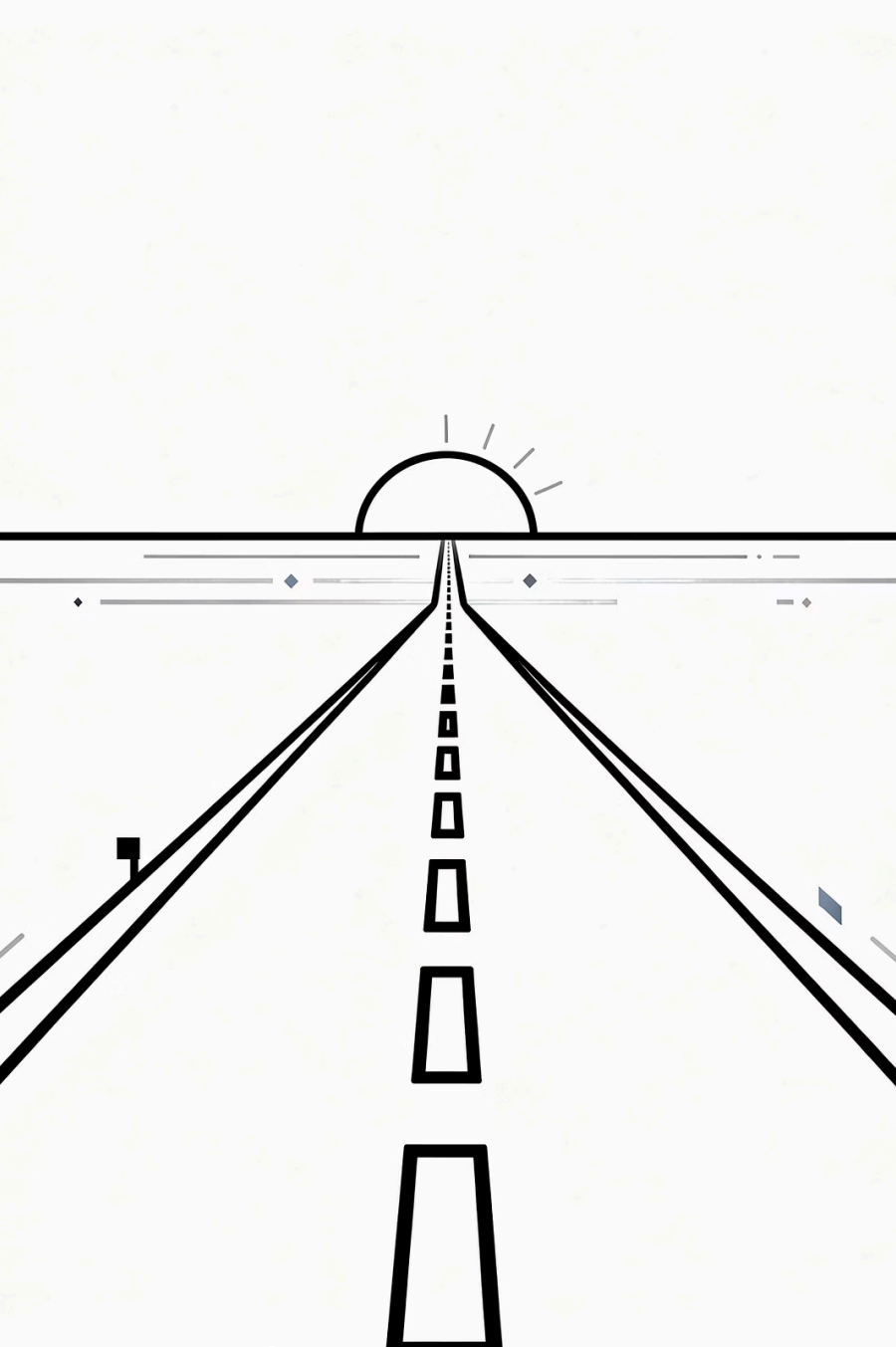
Spring Security

Authentification, autorisation, OAuth2, JWT et protection CSRF. Indispensable en contexte d'entreprise.



Tests avancés

Slices de test (@WebMvcTest, @DataJpaTest), TestContainers et WireMock pour les dépendances externes.



Conclusion

Vous avez parcouru les fondations de Spring Framework et Spring Boot — de l'architecture IoC à la maîtrise de l'injection de dépendances, en passant par la configuration avancée des beans.



Maîtriser

IoC, injection par constructeur et scopes des beans.



Pratiquer

Créer vos propres projets Spring Boot et expérimenter les approches de configuration.



Approfondir

Spring Data, Spring Security et Spring MVC sont les prochaines étapes naturelles.